PHASE

1

x86 Assembly and the Boot Process

Enough assembly to read and write kernel glue code, and the path a PC takes from power-on to the kernel's first instruction

Depends on Phase 0 complete: cross toolchain, NASM, QEMU, GDB

Delivers A NASM "phrase book" in the engineering journal, two assembly functions called from C, a 512-byte boot sector that prints a message, and a Multiboot kernel that both `qemu -kernel` and a GRUB ISO boot. `0xCAFEBABE` in `EAX` proves that it runs.

1 Scope and motivation

An operating system needs surprisingly little assembly. Kagan contains a few hundred lines of it in total. But those lines do the things C cannot do: set up the very first stack, load descriptor tables, save and restore registers on an interrupt, switch between processes, and drop into user mode. Code that cannot be read cannot be debugged. And the only way to see what is really happening when the kernel crashes is to read the machine state in terms of registers, the stack and instructions.

The second half of the phase traces how a PC goes from power-on to running kernel code. The path is followed once by hand, with a boot sector the BIOS loads directly, and then handed to GRUB, which Kagan uses from here on.

2 Background

2.1 Registers

The 32-bit x86 CPU has eight general-purpose registers. Each can be used as a whole (32 bits) or, for the first four, in smaller pieces that keep their names from the 8086 era (Figure 1). The remaining registers have fixed jobs, and several of them will get a phase of their own.



Figure 1: One register, four names. The same split applies to EBX, ECX and EDX.

Register(s)	Job	Where Kagan uses it
EAX, EBX, ECX, EDX	general purpose; EAX holds function return values	everywhere; EAX also carries the system call number (Phase 10)
ESI, EDI	general purpose; source and destination for string instructions	copying memory
ESP	stack pointer: address of the top of the stack	set up before any C runs (Phase 2); swapped on every context switch (Phase 8)
EBP	frame pointer: base of the current function's stack frame	walking the stack for backtraces (Phase 3)
EIP	instruction pointer: address of the next instruction	printed in every crash report
EFLAGS	condition flags (zero, carry, sign...) and the interrupt flag IF	`cli`/`sti` (Phase 4)
CS, DS, SS, ES, FS, GS	segment selectors	loaded from the kernel's own GDT (Phase 3)
CR0, CR2, CR3, CR4	control registers: protected mode, paging, fault address	paging (Phase 6)

2.2 A small instruction vocabulary

About twenty instructions cover nearly all the assembly in this project. NASM uses Intel syntax, where the destination comes first: `mov eax, 5` means `eax = 5`. Square brackets mean "the memory at this address": `mov eax, [ebx+4]` reads four bytes starting at address `ebx+4`. When the size is ambiguous, say it: `mov dword [eax], 0`.

Instruction	Meaning	C equivalent
`mov dst, src`	copy a value	`dst = src;`
`add`, `sub`, `inc`, `dec`	arithmetic; results set EFLAGS	`a += b; a--;`
`and`, `or`, `xor`, `not`, `shl`, `shr`	bitwise operations and shifts	`a &= m; a <<= 2;`
`cmp a, b` then `je`/`jne`/`jl`/`jg`/`jge`	compare (a subtraction that only sets flags), then jump on the result	`if (a == b) goto L;`
`test a, a` then `jz`	bitwise AND that only sets flags; common zero check	`if (a == 0) goto L;`
`jmp label`	unconditional jump	`goto label;`
`push x`, `pop x`	`esp -= 4; [esp] = x` and the reverse	(the call stack)
`call f`, `ret`	push the return address and jump; pop it and jump back	`f(); return;`
`lea eax, [ebx+ecx*4]`	compute an address without reading memory	`eax = &ebx[ecx];`
`in al, dx`, `out dx, al`	read or write an I/O port	(device access, Phase 2)
`cli`, `sti`, `hlt`	disable or enable interrupts; halt until the next interrupt	(Phase 4)
`int n`, `iret`	software interrupt; return from an interrupt handler	(Phases 3 and 10)

2.3 The stack and the cdecl calling convention

C code compiled for 32-bit x86 follows the **cdecl** convention, and Kagan assembly must follow it too whenever it calls C or is called from C. The caller pushes the arguments *right to left*, then executes `call`, which pushes the return address. Inside the callee, the first argument is therefore at `[esp+4]`, the second at `[esp+8]`, and so on. The return value goes in EAX. The callee may freely overwrite EAX, ECX and EDX (*caller-saved*), but must restore EBX, ESI, EDI, EBP and ESP before returning

(*callee-saved*). The caller removes the arguments afterwards. Most functions start by saving EBP and pointing it at their frame, so arguments sit at fixed offsets from EBP (Figure 2). The callee-saved list is the reason the context switch in Phase 8 only saves four registers.

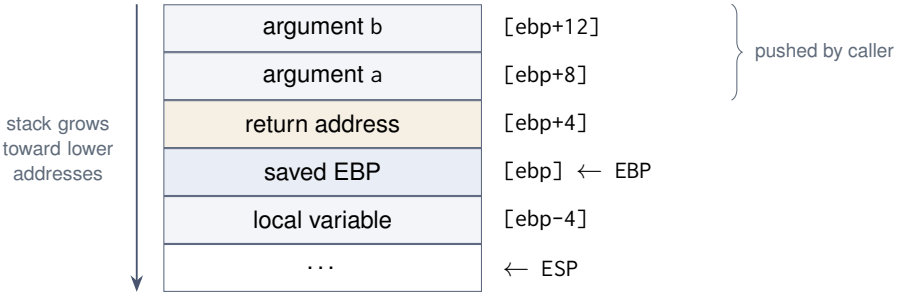


Figure 2: Stack frame inside `f(a, b)` after `push ebp; mov ebp, esp`. Higher addresses are at the top.

2.4 Real mode, protected mode, and how a PC boots

An x86 CPU wakes up in **real mode**, pretending to be a 1978 8086: 16-bit registers, addresses formed as `segment * 16 + offset`, one megabyte of reachable memory, no protection at all. The BIOS runs in real mode and so does the boot sector it loads. Operating systems switch to **protected mode** as soon as they can: 32-bit registers, a 4 GiB address space, privilege levels, and (later) paging. GRUB performs this switch. Figure 3 shows the whole chain.

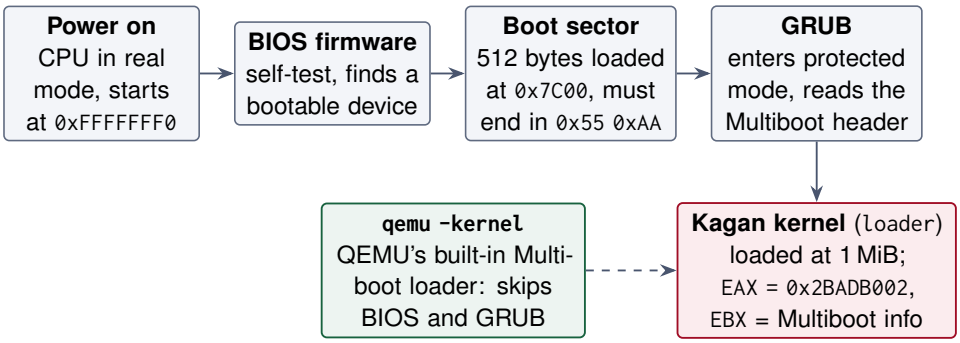


Figure 3: From power-on to the kernel. Task 1.3 supplies its own boot sector; from Task 1.4 on GRUB (or QEMU) does the loading.

The boot sector is loaded low in memory because the first megabyte is crowded (Figure 4). The kernel goes at 1 MiB, the first address above all of it.

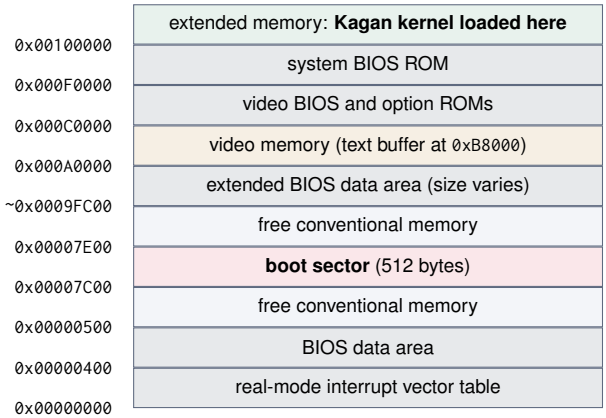


Figure 4: The first megabyte of a PC's physical memory (not to scale). Addresses mark the bottom of each region.

The Multiboot contract

When GRUB or QEMU jumps to the kernel, the Multiboot specification (§3.2, *Machine state*) guarantees: EAX holds the magic value `0x2BADB002`; EBX holds the physical address of the *Multiboot information* structure (memory size, memory map, boot modules); the CPU is in 32-bit protected mode with paging off; the segment registers describe flat 4 GiB segments; interrupts are disabled. It also states two things it does *not* provide. **ESP is undefined**, so the kernel must set up its own stack before calling any C function (Phase 2). And **the GDTR may be invalid**, so no segment register may be reloaded until the kernel has installed its own GDT (Phase 3). To be recognised, the kernel must contain a 12-byte Multiboot header (magic `0x1BADB002`, flags, checksum) aligned to 4 bytes within the first 8 KiB of the file.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	LOB	Ch. 2 *First Steps*: Booting, Hello Cafebabe (compiling, linking, GRUB, ISO, running)	12–17	The exact steps of Tasks 1.4–1.5.
PRIMARY	OSC	§2.9 *Building and Booting an Operating System*	92–95	Firmware, boot loaders and the boot sequence in general.
SECONDARY	MOS	§1.3.1 *Processors*; §1.3.6 *Booting the Computer*	21–24, 34–35	Registers, pipelines and privileged mode; the BIOS boot path.
SECONDARY	OSDI	§2.6.6 *Bootstrapping MINIX 3*	156–160	How a real x86 OS gets from the boot sector to its kernel.
OPTIONAL	OSDI	§4.6.2 *Segmentation with Paging: The Intel Pentium*	415–420	First look at real versus protected mode addressing.
OPTIONAL	OSC	§9.6 *Example: Intel 32- and 64-bit Architectures*	379–383	Preview of the address translation built in Phase 6.
OPTIONAL	LKP	Ch. 3, *Understanding the initramfs framework*, including the x86 boot process	111–117	How a production kernel uses GRUB and an initial RAM disk; contrast with Multiboot modules.

Three external references matter here. The **NASM manual**, chapters 2–3, covers syntax, labels and directives. The **Multiboot Specification 0.6.96**, section 3, defines the header and the machine state above. The **Intel SDM Volume 1**, chapter 3 (basic execution environment) and chapter 6 (procedure calls and the stack), is the authoritative version of Figure 2.

Prior art: MINIX 3

MINIX 3 does not use GRUB. It ships its own boot monitor, and OSDI §2.6.6 walks through what it does: load the kernel and the system processes into memory, switch to protected mode, and jump to the kernel's assembly entry point in `kernel/mpx386.s (06300)`. The first page of that file contains the stack setup and the jump into C (`kernel/start.c (06900)`), which is exactly what Phase 2 builds.

Linux, for comparison. LKD LKP Linux on x86 does not use Multiboot: GRUB loads it with the Linux boot protocol, together with an **initramfs** that becomes a temporary root file system until the real one is mounted.

4 Implementation

Task 1.1 Read compiler output systematically

Continue the phrase book started in Phase 0. For each C construct below, write the smallest function that uses it, compile with `i686-elf-gcc -O0 -S -masm=intel`, and annotate every instruction in the engineering journal: a local variable, an `if`, a `for` loop, an array index, a struct field access, a pointer dereference, and a call with two arguments. Then compile the same code with `-O2` and record what disappears.

Verify: For the two-argument call, the annotation identifies the two `push` instructions, the `call`, and the `add esp, 8` that removes the arguments.

Task 1.2 Write assembly that C can call

Install the 32-bit C runtime on the host (`sudo dnf install glibc-devel.i686 libgcc.i686`) so you can build 32-bit Linux test programs. Then write two functions in NASM and call them from a C test with `assert`. The first is a leaf function that reads its arguments straight from ESP. The second builds a proper frame and uses a callee-saved register, so it must preserve it.

playground/funcs.s

```
; int max(int a, int b)      cdecl: a at [esp+4], b at [esp+8], result in eax
global max
global sum_array
section .text
max:
    mov  eax, [esp + 4]     ; eax = a
    mov  ecx, [esp + 8]     ; ecx = b   (ecx is caller-saved: free to use)
    cmp  eax, ecx
    jge  .done              ; if (a >= b) return a
    mov  eax, ecx           ; else return b
.done:
    ret

; int sum_array(const int *a, int n)
sum_array:
    push ebp
    mov  ebp, esp           ; now [ebp+8] = a, [ebp+12] = n
    push ebx                ; ebx is callee-saved: we must restore it
    mov  ebx, [ebp + 8]     ; ebx = a
    mov  ecx, [ebp + 12]    ; ecx = n
    xor  eax, eax           ; sum = 0
.loop:
    test ecx, ecx
    jz   .done
    add  eax, [ebx]         ; sum += *a
    add  ebx, 4             ; a++   (an int is 4 bytes)
    dec  ecx
    jmp  .loop
.done:
    pop  ebx
    pop  ebp
    ret
```

terminal

```
nasm -f elf32 playground/funcs.s -o build/funcs.o
gcc -m32 -g playground/test_funcs.c build/funcs.o -o build/test_funcs && ./build/test_funcs
```

Verify: The asserts pass. Then step through `sum_array` in GDB with `layout asm`, `stepi` and `info registers`, and watch ESP change on every `push` and `pop`.

Task 1.3 A boot sector that says hello

This is a deliberate detour: the first 512 bytes a PC executes from disk, in 16-bit real mode, using a BIOS service to print. Kagan itself does not use this code; GRUB replaces it in the next task.

playground/boot.s

```
bits 16                     ; the CPU is in 16-bit real mode
org  0x7C00                 ; the BIOS loads us at this address

start:
    xor  ax, ax
    mov  ds, ax             ; data segment = 0, so [si] means address si
    mov  si, msg
.next:
    lodsb                   ; al = [ds:si], then si = si + 1
    test al, al
    jz   .halt              ; stop at the terminating zero
    mov  ah, 0x0E           ; BIOS "teletype output" function
    int  0x10               ; ask the BIOS video service to print al
    jmp  .next
.halt:
    cli
    hlt
    jmp  .halt

msg: db "Merhaba, Kagan!", 0

times 510 - ($ - $$) db 0   ; pad with zeros up to byte 510
dw 0xAA55                   ; boot signature: bytes 0x55, 0xAA
```

terminal

```
nasm -f bin playground/boot.s -o build/boot.bin
ls -l build/boot.bin            # exactly 512 bytes
hexdump -C build/boot.bin | tail -n 2   # last bytes: 55 aa
qemu-system-i386 -drive format=raw,file=build/boot.bin
```

Verify: QEMU prints the message after the SeaBIOS banner. Delete the `dw 0xAA55` line and confirm that the BIOS now refuses to boot the disk.

Task 1.4 Hello Cafebabe: a Multiboot kernel

The actual Kagan entry point, following LOB chapter 2. The kernel only loads a recognisable value into EAX and spins, so inspecting the register proves the code is running. Put the Multiboot header in its own section and have the linker place it first, which guarantees it lands in the first 8 KiB of the file.

boot/loader.s

```
global loader                   ; the entry symbol, named in link.ld

MAGIC_NUMBER equ 0x1BADB002     ; tells GRUB this is a Multiboot kernel
FLAGS        equ 0x0            ; no extra requests yet (Phase 2 changes this)
CHECKSUM     equ -(MAGIC_NUMBER + FLAGS)   ; magic + flags + checksum == 0

section .multiboot
align 4
    dd MAGIC_NUMBER
    dd FLAGS
    dd CHECKSUM

section .text
```

```
loader:
    mov eax, 0xCAFEBABE         ; a value you will recognise in a register dump
.loop:
    jmp .loop                   ; spin forever
```

link.ld

```
ENTRY(loader)                   /* where execution starts */
SECTIONS
{
    . = 0x00100000;             /* load everything from 1 MiB upwards */

    .text ALIGN(0x1000) : {
        *(.multiboot)           /* Multiboot header first */
        *(.text)
    }
    .rodata ALIGN(0x1000) : { *(.rodata*) }
    .data   ALIGN(0x1000) : { *(.data) }
    .bss    ALIGN(0x1000) : { *(COMMON) *(.bss) }
}
```

Build it with `nasm -f elf32 boot/loader.s -o build/loader.o`, then `i686-elf-ld -T link.ld -o build/kernel.elf build/loader.o`. Run it with QEMU's built-in Multiboot loader and open the monitor on the terminal:

terminal

```
grub2-file --is-x86-multiboot build/kernel.elf && echo "multiboot header OK"
qemu-system-i386 -kernel build/kernel.elf -monitor stdio
(qemu) info registers
```

Verify: `info registers` shows `EAX=cafebabe` and an `EIP` just above `0x00100000`.

Task 1.5 Boot through a real GRUB ISO

LOB builds its ISO with GRUB Legacy's `stage2_eltorito`. On a current Fedora system, GRUB 2 does the whole job with `grub2-mkrescue`. Create a GRUB configuration and let the tool assemble a bootable CD image around the kernel:

boot/grub.cfg

```
set timeout=0
set default=0

menuentry "Kagan" {
    multiboot /boot/kernel.elf
    boot
}
```

terminal

```
mkdir -p build/iso/boot/grub
cp build/kernel.elf build/iso/boot/
cp boot/grub.cfg build/iso/boot/grub/
grub2-mkrescue -o build/kagan.iso build/iso
qemu-system-i386 -cdrom build/kagan.iso -monitor stdio
```

Verify: Same result as Task 1.4: `EAX=cafebabe`. The kernel now boots exactly as on a real PC.

Task 1.6 A Makefile for the kernel

Typing these commands by hand does not scale. Write a Makefile with the targets `all`, `run`, `iso`, `debug` and `clean`. Recipe lines must start with a real Tab character.

Makefile

```
AS    := nasm
LD    := i686-elf-ld
BUILD := build
OBJS  := $(BUILD)/loader.o

all: $(BUILD)/kernel.elf

$(BUILD)/kernel.elf: $(OBJS) link.ld
    $(LD) -T link.ld -o $@ $(OBJS)

$(BUILD)/%.o: boot/%.s
    @mkdir -p $(BUILD)
    $(AS) -f elf32 $< -o $@

run: $(BUILD)/kernel.elf
    qemu-system-i386 -kernel $< -monitor stdio

iso: $(BUILD)/kernel.elf boot/grub.cfg
    mkdir -p $(BUILD)/iso/boot/grub
    cp $< $(BUILD)/iso/boot/kernel.elf
    cp boot/grub.cfg $(BUILD)/iso/boot/grub/
    grub2-mkrescue -o $(BUILD)/kagan.iso $(BUILD)/iso

debug: $(BUILD)/kernel.elf
    qemu-system-i386 -kernel $< -s -S &
    gdb $< -ex "target remote localhost:1234" -ex "break loader" -ex "continue"

clean:
    rm -rf $(BUILD)

.PHONY: all run iso debug clean
```

Verify: `make clean && make run` rebuilds and boots from scratch.

Task 1.7 A kernel debugging session

Run `make debug`. QEMU starts frozen (`-S`) and waits for GDB on port 1234 (`-s`). GDB stops at `loader`. Try `info registers`, `stepi`, `x/4xw $esp` and `p/x $eax`, and use `layout asm` for a live disassembly view. Every later phase relies on this workflow, and it is easiest to establish while the kernel is still two instructions long.

Verify: After one `stepi`, `p/x $eax` prints `0xcafebabe`.

5 Testing and debugging

Symptom	Likely cause	Resolution
QEMU or BIOS: “No bootable device” with the boot sector	File is not exactly 512 bytes, or the last two bytes are not `55 AA`	Check with `ls -l` and `hexdump -C`; keep the `times` padding line.
Boot sector prints garbage	Missing `org 0x7C00` or `bits 16`, or DS not zero	All label addresses depend on `org`; set DS before `lodsb`.
GRUB: “no multiboot header found”	Header not in the first 8 KiB, not 4-byte aligned, or a bad checksum	Keep `*(.multiboot)` first in `.text`; run `grub2-file --is-x86-multiboot`.
QEMU `-kernel`: “Error loading uncompressed kernel without PVH ELF Note”	QEMU found no Multiboot header, or the ELF is 64-bit	Same checks; make sure the build used `nasm -f elf32` and the `i686-elf` linker.

Symptom	Likely cause	Resolution
`objdump` output looks backwards	AT&T syntax (source first)	Use `objdump -d -M intel` to match NASM.
The C test crashes after calling `sum_array`	A callee-saved register (EBX) was not restored, or the stack is unbalanced	Every `push` needs a matching `pop` before `ret`.

6 Exit criteria

Exit criteria: The kernel boots

- ☐ `max` and `sum_array` in NASM pass their C tests and have been stepped through in GDB.
- ☐ The boot sector prints "Merhaba, Kagan!" in QEMU and is exactly 512 bytes.
- ☐ `kernel.elf` boots with `make run` and with the GRUB ISO; `EAX=0xCAFEBABE` in both.
- ☐ `make debug` stops at `loader` and single-stepping works.
- ☐ The cdecl stack frame for a two-argument call is documented in the engineering journal.

Review questions

1. Why is the boot sector loaded at `0x7C00`, and why must it end with `0x55 0xAA`?
2. What does the Multiboot checksum guarantee? Compute it for `FLAGS = 0x3`.
3. Right after `call f`, what is at `[esp]`? Inside `f`, after `push ebp; mov ebp, esp`, where is the first argument?
4. Which registers must a cdecl function preserve, and which may it destroy?
5. Why is the kernel linked at 1 MiB rather than at address 0?
6. What two things does the Multiboot contract explicitly *not* set up?
7. How does a real-mode address like `0x07C0:0x0000` map to a physical address?

7 Extensions

Extend the boot sector: read a key with BIOS `int 0x16` and echo it, write a routine that prints a register in hex, or load a second sector from disk with `int 0x13`. That last one is the first step of a custom boot loader. A custom boot loader is out of scope for this project, so this work should stay time-boxed. For deeper assembly work, the Intel SDM Volume 1 chapter on procedure calls is the reference; write `memcpy` and `strlen` in NASM and test them against the C versions from Phase 0.

Next: **Phase 2 Getting to C: Screen, Serial and kprintf** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and **OSTEP** are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.